

Цели и задачи

Цель работы

Целью лабораторной работы является практическое освоение информации, полученной при изучении курса «Компьютерное моделирование в математической физике» по теме «Численное решение стационарного уравнения Шрёдингера», а также развития алгоритмического мышления и приобретения опыта использования знаний и навыков по математике, численным методам и программированию для решения прикладных задач физико-технического характера.

Задачи работы

Проблема: электрон находится в одномерной потенциальной яме с бесконечными стенками

$$v(x) = \begin{cases} 0, & x \in [-\frac{L}{2}, 0] \\ -1, & x \in [-L, -\frac{L}{2}] \forall x \in [0, L] \\ \infty, & x \leq -L \vee x \geq [0, L] \end{cases}$$

где $V_0 = 20\text{эВ}$, $L = 2\text{А}$.

Поставлена задача, используя метод пристрелки, найти энергию, нормированную волновую функцию и плотность вероятности для основного и 1-го возбужденного состояний. После проведенных расчетов требуется для каждого из вычисленных состояний найти квантовомеханические средние $\langle x \rangle$ и $\langle x^2 \rangle$.

Одномерное стационарное уравнение Шрёдингера.

Математический формализм. Общие свойства решений.

Одномерное стационарное уравнение Шрёдингера

$$\hat{H}\psi(x) = E\psi(x)$$

с математической точки зрения представляет собой задачу определения собственных значений E и собственных функций $\psi(x)$ оператора Гамильтона $\hat{H}$. Для частицы с массой m , находящейся в потенциальном поле (называемом также потенциалом) $U(x)$, оператор Гамильтона имеет вид

$$\hat{H} = \hat{T} + U(x)$$

Собственное значение оператора Гамильтона имеет смысл энергии соответствующей изолированной квантовой системы. Собственные функции называются волновыми функциями. Волновая функция однозначна и непрерывна во всем пространстве. Непрерывность волновой функции и ее 1-ой производной сохраняется и при обращении $U(x)$ в ∞ в некоторой области пространства. В такую область частицы вообще не может проникнуть, то есть в этой области, а также на её границе (что следует из непрерывности функции), $\psi(x) = 0$.

Оценим нижнюю границу энергетического спектра. Пусть минимальное значение потенциальной энергии равно $U_{\min}$. Очевидно, что $\langle T \rangle \geq 0$ и $\langle U \rangle \geq U_{\min}$. Потому, из уравнения (2) следует, что

$$E = \langle H \rangle = \int_{-\infty}^{+\infty} \psi^*(x) \hat{H} \psi(x) dx = \langle T \rangle + \langle U \rangle > U_{\min}$$

Особый практический интерес представляет случай, когда

$$\lim_{x \rightarrow \pm\infty} U(x) = 0$$

Потенциал такого типа (называемый также потенциальной ямой) изображен на рис. 1. Для данной $U(x)$ свойства решений уравнения Шрёдингера зависят от знака собственного значения E .

Рассмотрим случай, когда $E < 0$. Частица с отрицательной энергией совершает финитное движение. Оператор Гамильтона имеет дискретный спектр, то есть собственные значения и соответствующие собственные функции можно снабдить номерами (называемые квантовыми числами). При $E < 0$ уравнение (2) приобретает вид

$$\hat{H}\psi_k(x) = E_k\psi_k(x)$$

Квантовое состояние, обладающее наименьшей энергией, называется основным. Остальные состояния называются возбужденными состояниями. Квантовые состояния дискретного спектра называют связанными состояниями. Частица, находящаяся в связанном состоянии, не способна уйти на бесконечность. То есть, плотность вероятности $|\psi_k(x)|^2 \rightarrow 0$ при $x \rightarrow \pm\infty$, но на всех конечных расстояниях $\neq 0$. В силу линейности стационарного уравнения Шрёдингера, волновые функции математически определены с точностью до постоянного множителя. Однако, из физических соображений, волновые функции должны быть нормированы следующим образом:

$$\int_{-\infty}^{+\infty} |\psi_k(x)|^2 dx = 1$$

Перейдем к случаю, когда $E > 0$. Частица совершает инфинитное движение. Оператор Гамильтона имеет непрерывный спектр собственных значений. Квантовые состояния непрерывного спектра называют несвязанными состояниями. Частица, находящаяся в несвязанном состоянии, способна уйти в бесконечность.

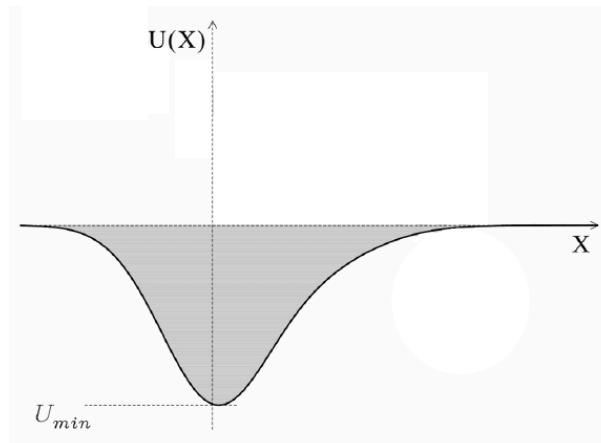


Figure 1: Потенциальное поле $U(x)$

Поскольку мы занимаемся численным моделированием квантовых состояний только для дискретного спектра, необходима *осцилляционная теорема*

Осцилляционная теорема

Упорядочим собственные значения оператора Гамильтона в порядке возрастания, нумеруя энергию основного состояния индексом «0»: $E_0, E_1, E_2, \dots, E_k, \dots$. Тогда волновая функция k -го состояния $\psi_k(x)$ будет иметь k узлов (то есть пересечений с осью абсцисс). Исключения: области, в которых потенциальная функция бесконечна.

Волновые функции, представленные на рис. 2, хорошо иллюстрируют осцилляционную теорему.

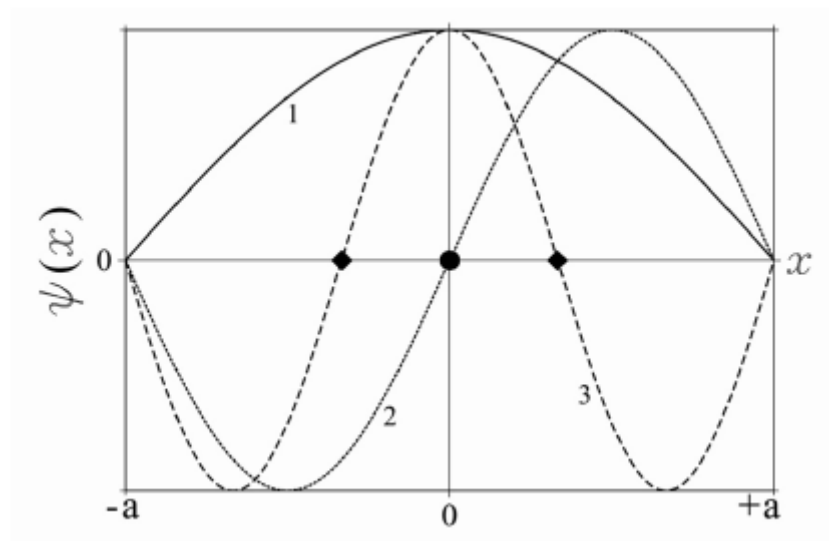


Figure 2: Прямоугольная потенциальная яма с бесконечными стенками: волновые функции $\psi_1(x), \psi_2(x)$ и $\psi_3(x)$

Для решения уравнения Шрёдингера удобно использовать атомные единицы Хартри. В этих единицах уравнение (6), предполагая, что $m = m_e$, приобретает вид

$$\left[-\frac{1}{2} \frac{d^2}{dx^2} + U(x) \right] \psi(x) = E \psi(x)$$

Преобразуем (8) к форме

$$d^2 \frac{\psi(x)}{dx^2} + q(E, x) \psi(x) = 0$$

, где

$$q(E, x) = 2[E - U(x)]$$

Формула для нахождения квантомеханического среднего $\langle x \rangle$ имеет следующий вид:

$$\langle x \rangle = \int_a^b \psi_n^*(x) x \psi(x) dx = \int_a^b x \psi^2(x) dx$$

Метод пристрелки. Алгоритм

Решение стационарного уравнения Шрёдингера сводится к нахождению собственных значений и собственных функций оператора Гамильтона, которые будем вычислять по изложенному ниже алгоритму.

Алгоритм.

Нас будут интересовать только основное и низковозбужденное состояния электрона для потенциального профиля (1).

Поскольку для собственных значений известна оценка снизу (4), то удобно начинать с вычисления энергии и волновой функции основного состояния. Оценим грубо энергию основного состояния. Подставим значение этой энергии в уравнение (8). Это уравнение теперь становится обыкновенным дифференциальным уравнением 2-го порядка с граничными условиями $\psi(a) = \psi(b) = 0$. Вместо ОДУ 2-го порядка (8) рассматриваем эквивалентную систему 2-х ОДУ 1-го порядка:

$$\begin{cases} \frac{d\psi(x)}{dx} = \psi_1(x) \\ \frac{d\psi_1(x)}{dx} = -q(E, x)\psi(x) \end{cases}$$

Для нахождения точного собственного значения оператора Гамильтона процедуры интегрирования «вперед» и «назад» должны приводить к одинаковым результатам (с точностью до постоянного множителя и ошибок округления). При неравенстве E собственному значению, эти процедуры дают различные функции, отношение которых в различных узлах сетки не равно постоянной величине. Для оценки близости E к собственному значению будем вычислять разность производных волновых функций, полученных интегрированием «вперед» и «назад», в некотором узле сетки x_m

$$f(E) = \frac{d\psi_{>}(x)}{dx} \Big|_{x_m} - \frac{d\psi_{<}(x)}{dx} \Big|_{x_m}$$

Здесь $\psi_{>}(x)$ и $\psi_{<}(x)$ - волновые функции, полученные интегрированием «вперед» и «назад» соответственно; x_m - узел сшивки производных. Перед вычислением (15) необходимо масштабировать функции $\psi_{>}(x)$ и $\psi_{<}(x)$ так, чтобы $\psi_{>}(x_m) = \psi_{<}(x_m)$. Такое масштабирование будем называть математической нормировкой. Вычисление f позволяет организовать процедуру поиска собственного значения методом бисекции (см. блок-схему на рис. 3). Выберем нулевое приближение к энергии основного состояния следующим образом: $E(0) = U_{\min} + \delta$, где δ малая величина ($\delta > 0$), и вычислим соответствующее $f(0)$ по формуле (15). Верхний индекс в скобках означает номер итерации. Далее, будем увеличивать энергию с шагом ΔE до тех пор, пока величины $f(i)$ на двух соседних шагах i и $i - 1$ не будут иметь разные знаки. Если шаг ΔE был меньше, чем разность энергий соседних уровней (собственных значений) в актуальном диапазоне изменения E , то можно быть уверенным, что искомое собственное значение $\in [E(i - 1), E(i)]$. Далее для уточнения собственного значения с наперед заданной точностью ϵ используется метод бисекции.

Задача Коши для дифференциального уравнения (12) в программе решается с использованием пакетов numru и scipy, в частности с использованием функции `scipy.integrate.odeint`. В финальной части программы осуществляется вывод графиков функций $f(x)$, $\psi(x)$, $|\psi(x)|^2$, потенциальной ямы $U(x)$ согласно варианту, найденных энергий $E_0, E_1 (n = 1)$ и найденных квантомеханических средних $\langle x \rangle, \langle x^2 \rangle$.

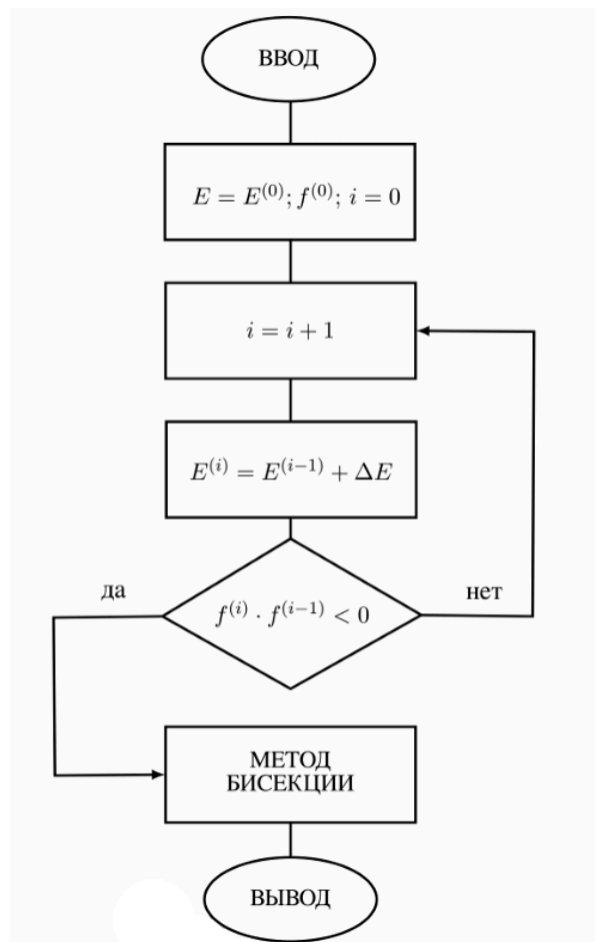


Figure 3: Блок-схема процедуры поиска собственного значения

Программная реализация алгоритма

В строках 25-31 файла main.py задаётся потенциальная функция $U(x)$ по формуле (1).

В строках 34-35 файла main.py определяется вспомогательная функция $q(E, x)$, заданная формулой (10).

В строках 39-54 файла main.py происходит интегрирование «вперед» и интегрирование «назад» (члены правой части формулы (13)).

В строках 4-44 файла qm.py задаются функции для отыскания квантомеханических средних $\langle p(x) \rangle$ и $\langle p(x^2) \rangle$. Для их нахождения применяется метод Симпсона.

В строках 58-100 файла main.py вычисляется разность производных в узле сшивки — это функция $f(E)$ (формула (13)).

В строках 103-125 файла main.py применяется метод бисекции для решения уравнения $f(E) = 0$.

В строках 128-166 файла main.py задается функция вывода графиков функций $\psi(x)$, $f(x)$, $U(x)$, $|\psi(x)|^2$.

В строках 170-221 файла main.py определяются необходимые переменные.

В строках 223-259 файла main.py вызывается главный цикл программы — цикл поиска простых корней $f(E)$ на отрезке $[e1, e2]$, то есть собственных значений оператора Гамильтона $\hat{H}$.

В конце программы, в строках 262-295 файла main.py, выводятся результаты моделирования в консоль и текстовый файл.

Результаты численных экспериментов и их анализ

Ниже представлены некоторые графические результаты работы программы для решения стационарного уравнения Шрёдингера методом пристрелки. Как видно по графикам, осцилляционная теорема выполняется. Поскольку потенциальная функция несимметричная, квантомеханические средние импульсы нулевые. $\langle p_x \rangle = 0$, т.к. волновая функция смещена в правую часть ямы, $\langle p_x^2 \rangle = 0$ - волновая функция смещена в левую часть ямы, что и иллюстрируют полученные графики.

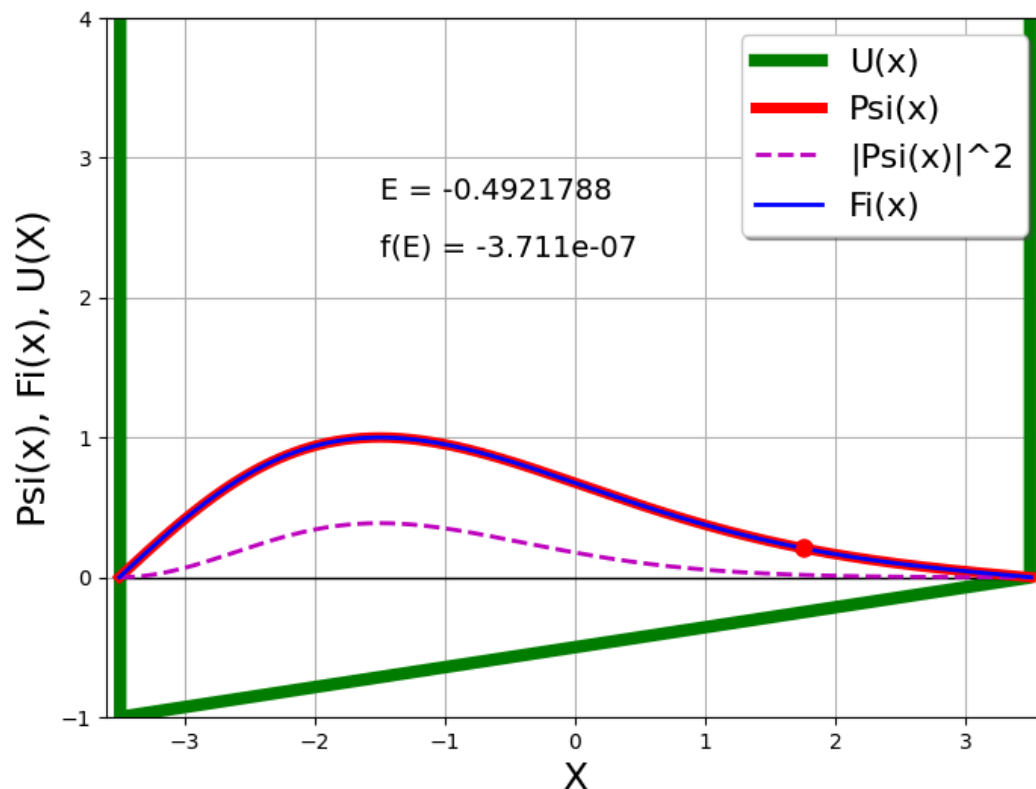
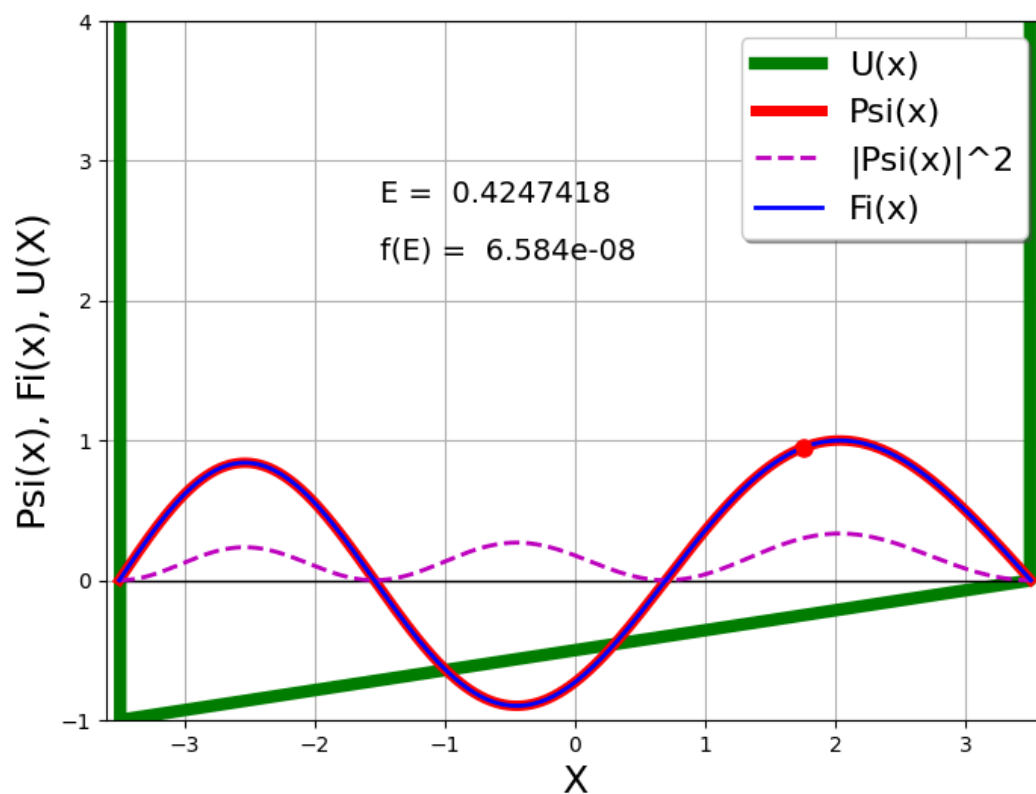


Figure 4: Волновая функция и плотность вероятности основного состояния



ЭНЕРГИИ

k	E, a.u.	E, eV
0	-0.4921788	-13.3926762
2	0.4247418	11.5576491

КВАНТОМЕХАНИЧЕСКИЕ СРЕДНИЕ

k	$\langle x \rangle$	$\langle x^2 \rangle$
0	0.0000000-0.0000012j	0.3448626
2	0.0000000-0.0000012j	1.7793792

Приложение 1. Компьютерный код

Листинг файла main.py

```
"""
Вычисление собственных значений и собственных функций
оператора Гамильтона методом пристрелки.
Одномерная потенциальная яма с бесконечными стенками.
Атомные единицы Хартри.
Версия 2.
"""

import numpy as np
from scipy.integrate import odeint
# from scipy.misc import derivative
from scipy.optimize import approx_fprime
from scipy.interpolate import interp1d
from scipy.integrate import simpson

from qma import qua_mech_ave_p, qua_mech_ave_p2
import matplotlib.pyplot as plt
global r, n, Psi, Fi, X, XX
plt.rcParams['figure.figsize'] = [8, 6]

LST = open("schrodinger-2b.txt", "wt")

# потенциальная функция, рис. 3
# (на рис.3. "a" соответствует "L")
def U(x):
    # return float(U0 if abs(x) < L else W)
    if (x > -L) and (x < L):
        y = (-1 + (x + L) / (2 * L))
    else:
        y = 9999999
    return y

# функция, ф-ла (13)
def q(e, x):
    return 2.0*(e-U(x))

# вычисление правых частей системы ОДУ 1-го порядка
# (интегрирование "вперёд")
def system1(cond1, X):
    global eee
    Y0, Y1 = cond1[0], cond1[1]
    dY0dX = Y1
    dY1dX = - q(eee, X)*Y0
    return [dY0dX, dY1dX]

# вычисление правых частей системы ОДУ 1-го порядка
# (интегрирование "назад")
```

```

def system2(cond2, X):
    global eee
    Z0, Z1 = cond2[0], cond2[1]
    dZ0dX = Z1
    dZ1dX = - q(eee, X)*Z0
    return [dZ0dX, dZ1dX]

# вычисление разности производных в узле сшивки,
# формула (18)
def f_fun(e):
    global r, n, Psi, Fi, X, XX, eee
    eee = e
    """
    Решение задачи Коши ("Вперёд")
    Psi1(x)/dx = - q(e, x) * Psi(x);
    dPsi(x)/dx = Psi1(x);
    Psi(A) = 0.0
    Psi1(A) = 1.0
    """

    cond1 = [0.0, 1.0]
    sol1 = odeint(system1, cond1, X)
    Psi, Psi1 = sol1[:, 0], sol1[:, 1]
    """
    Решение задачи Коши ("назад")
    Psi1(x)/dx = - q(e, x) * Psi(x);
    dPsi(x)/dx = Psi1(x);
    Psi(B) = 0.0
    Psi1(B) = 1.0
    """

    cond2 = [0.0, 1.0]
    sol2 = odeint(system2, cond2, XX)
    Fi, Fi1 = sol2[:, 0], sol2[:, 1]
    # поиск максимального по величине элемента Psi
    p1 = np.abs(Psi).max()
    p2 = np.abs(Psi).min()
    big = p1 if p1 > p2 else p2
    # масштабирование Psi
    Psi[:] = Psi[:]/big
    # математическая нормировка Fi для
    # достижения равенства F[rr]=Psi[r]
    coef = Psi[r]/Fi[rr]
    Fi[:] = coef*Fi[:]
    # вычисление f(E) для узла сшивки, формула(18)
    curve1 = interp1d(X, Psi, kind='cubic')
    curve2 = interp1d(XX, Fi, kind='cubic')
    # der1 = derivative(curve1, X[r], dx=1.e-6)
    der1 = approx_fprime(X[r], curve1, epsilon=1.e-6)
    # der2 = derivative(curve1, XX[rr], dx=1.e-6)
    der2 = approx_fprime(XX[rr], curve2, epsilon=1.e-6)
    # print(f"debugprint: 1 {der1} -- 2 {der2}")
    f=der1[0]-der2[0]

```

```

    return f

#функция для решения уравнения  $f(E) = 0$  методом бисекций
def m_bis(x1, x2, tol):
    global r, n
    # Проверка наличия корня на интервале
    if f_fun(e=x2) * f_fun(e=x1) > 0.0:
        print("ERROR::no_root!!!")
        print("x1 =", x1)
        print("x2 =", x2)
        print("f_fun(x1) =", f_fun(e=x1))
        print("f_fun(x2) =", f_fun(e=x2))
        exit()
    # Итерационный процесс метода бисекции
    while abs(x2 - x1) > tol:
        xr = (x1 + x2) / 2.0
        if f_fun(e=x2) * f_fun(e=xr) < 0.0:
            x1 = xr
        else:
            x2 = xr
        if f_fun(e=x1) * f_fun(e=xr) < 0.0:
            x2 = xr
        else:
            x1 = xr
    # Возвращаем приближенное значение корня
    return (x1 + x2) / 2.0

# Функция для вывода графика волновых функций и потенциала
def plotting_wf(e):
    global r, n, Psi, Fi, X, XX
    A_edge = A - 0.1 # расширяем слева
    B_edge = B + 0.1 # расширяем справа
    ff = f_fun(e)
    # Установка границ графика
    plt.axis([A_edge, B_edge, -1, W])
    # Вычисление потенциала
    Upot = np.array([U(X[i]) for i in np.arange(n)])
    # Построение графика потенциала
    plt.plot(X, Upot, 'g-', linewidth=6.0, label="U(x)")
    # Нулевая линия
    Zero = np.zeros(n, dtype=float)
    plt.plot(X, Zero, 'k-', linewidth=1.0)
    # Волновые функции
    plt.plot(X, Psi, 'r-', linewidth=5.0, label="Psi(x)")

    plt.plot([X[r]], [Psi[r]], color = 'red', marker = 'o', markersize = 8)
    plt.plot(X, prob_density, 'm--', linewidth = 2.0, label = "|Psi(x)|^2")

    plt.plot(XX, Fi, 'b-', linewidth=2.0, label="Fi(x)")

    # Настройка осей

```

```

plt.xlabel("X", fontsize=18, color="k")
plt.ylabel("Psi(x), Fi(x), U(X)", fontsize=18, color="k")
plt.grid(True)
# Легенда
plt.legend(fontsize=16, shadow=True, fancybox=True, loc='upper right')
# Отметка специальной точки
plt.plot([X[r]], [Psi[r]], color='red', marker='o', markersize=7)
# Текст с информацией об энергии и функции
string1 = "E = " + format(e, "10.7f")
string2 = "f(E) = " + format(ff, "10.3e")
plt.text(-1.5, 2.7, string1, fontsize=14, color='black')
plt.text(-1.5, 2.3, string2, fontsize=14, color="black")
# Сохранение в файл
name = "schrodinger-2b-" + str(ngr) + ".pdf"
plt.savefig(name, dpi=300)
plt.show()

# Задание отрезка [A, B] (края ямы)
# L = 2.0
L = 3.5
A = -L
B = +L

# Кол-во узлов сетки на [A, B]
n = 1001 # нечетное целое число
print("n =", n)
print("n =", n, file=LST)

# Минимальное значение потенциальной функции
U0 = -1.0

# Максимальное значение потенциальной функции на графике
W = 4.0

# x-координаты узлов сетки
X = np.linspace(A, B, n) # для интегрирования "спереди"
XX = np.linspace(B, A, n) # для интегрирования "назад"

# Номер узла сетки
r = (n - 1) * 3 // 4 # для Psi
rr = n - r - 1      # для Fi

print("r =", r)
print("r =", r, file=LST)
print("rr =", rr)
print("rr =", rr, file=LST)
print("X[r] =", X[r])
print("X[r] =", X[r], file=LST)
print("XX[rr] =", XX[rr])
print("XX[rr] =", XX[rr], file=LST)
# Поиск корней f(E)

```

```

e1 = U0 + 0.05
e2 = 3.0
print("e1 =", e1, "    e2 =", e2)
print("e1 =", e1, "    e2 =", e2, file=LST)

ne = 201
print("ne =", ne)
print("ne =", ne, file=LST)

prob_density = np.zeros(n)

ee = np.linspace(e1, e2, ne)
af = np.zeros(ne, dtype=float)
porog = 5.0
tol = 1.0e-7
energy = []
ngr = 0

qma_p_l = []
qma_pp_l = []
# Цикл поиска корней f(E) на отрезке [e1, e2]
for i in np.arange(ne):
    e = ee[i]
    af[i] = f_fun(e)

    stroka = "i = {:3d}    e = {:8.5f}    f(e) = {:12.5e}"
    print(stroka.format(i, e, af[i]))
    print(stroka.format(i, e, af[i]), file=LST)

    if i > 0:
        Log1 = af[i] * af[i-1] < 0.0 # проверка смены знака
        Log2 = np.abs(af[i] - af[i-1]) < porog # проверка непрерывности

        if Log1 and Log2:
            energy1 = ee[i-1]
            energy2 = ee[i]
            eval = m_bis(energy1, energy2, tol) # уточнение корня методом бисекции

            print("eval = {:12.5e}".format(eval))

            # Построение графика волновой функции
            energy.append(eval)

            norm_factor = simpson(Psi**2, X)
            if norm_factor != 0:
                Psi[:] = Psi / np.sqrt(norm_factor)
            prob_density = Psi**2

            dummy = plotting_wf(eval)

            Psi_copy = Psi.copy()

```

```

p_qma = qua_mech_ave_p(Psi_copy, X)
pp_qma = qua_mech_ave_p2(Psi_copy, X)

qma_p_l.append(p_qma)
qma_pp_l.append(pp_qma)

ngr += 1

# Вывод значений корней уравнения  $f(E) = 0$ 
print(qma_p_l)
tt = "-----"
print(tt)
print(tt, file = LST)
print("          ЭНЕРГИИ")
print("          ЭНЕРГИИ", file = LST)
print(tt)
print(tt, file = LST)
print("k          E, a.u.          E, eV")
print("k          E, a.u.          E, eV", file = LST)
print(tt)
print(tt, file = LST)
for k in range(ngr):
    print("{:1d}      {:12.7f}      {:12.7f}".format(k, energy[k], 27.211*energy[k]))
    print("{:1d}      {:12.7f}      {:12.7f}".format(k, energy[k], 27.211*energy[k]),
file = LST)

print(" ")
print(" ", file = LST)
print(" ")
print(" ", file = LST)

print(tt)
print(tt, file = LST)
print("  КВАНТОМЕХАНИЧЕСКИЕ СРЕДНИЕ")
print("  КВАНТОМЕХАНИЧЕСКИЕ СРЕДНИЕ", file = LST)
print(tt)
print(tt, file = LST)
print("k          <x>          <x^2>")
print("k          <x>          <x^2>", file = LST)
print(tt)
print(tt, file = LST)
for k in range(ngr):
    print("{:1d}      {:12.7f}      {:12.7f}".format(k, qma_p_l[k], qma_pp_l[k]))
    print("{:1d}      {:12.7f}      {:12.7f}".format(k, qma_p_l[k], qma_pp_l[k]), file =
LST)

fmax = +10.0
fmin = -10.0

```

Листинг файла qma.py


```

import numpy as np
from scipy.integrate import simpson

def qua_mech_ave_p(Psi, X):
    """Вычисление квантовомеханического среднего <p>"""
    # hbar = 1 в атомных единицах, или hbar = 1.0545718e-34 в СИ
    hbar = 1.0 # для простоты, можно изменить

    # Вычисляем производную волновой функции
    dPsi_dx = np.gradient(Psi, X)

    # Вычисляем числитель:  $\int \psi^* (-i\hbar d\psi/dx) dx$ 
    # Для комплексных функций нужно комплексное сопряжение
    if np.iscomplexobj(Psi):
        num = simpson(np.conj(Psi) * (-1j * hbar * dPsi_dx), X)
        norm = simpson(np.abs(Psi)**2, X)
    else:
        # Для вещественных функций
        num = simpson(Psi * (-1j * hbar * dPsi_dx), X)
        norm = simpson(Psi**2, X)

    # Мнимая часть должна быть малой (проверка)
    result = num / norm if abs(norm) > 1e-12 else 0
    return result

def qua_mech_ave_p2(Psi, X):
    """Вычисление квантовомеханического среднего <p^2>"""
    hbar = 1.0

    # Вычисляем вторую производную
    dPsi_dx = np.gradient(Psi, X)
    d2Psi_dx2 = np.gradient(dPsi_dx, X)

    # Вычисляем числитель:  $\int \psi^* (-\hbar^2 d^2\psi/dx^2) dx$ 
    if np.iscomplexobj(Psi):
        num = simpson(np.conj(Psi) * (-hbar**2 * d2Psi_dx2), X)
        norm = simpson(np.abs(Psi)**2, X)
    else:
        num = simpson(Psi * (-hbar**2 * d2Psi_dx2), X)
        norm = simpson(Psi**2, X)

    # Результат должен быть вещественным
    result = num / norm if abs(norm) > 1e-12 else 0
    return np.real(result) # Берем вещественную часть

```

1. Фаддеев Л. Д. Что такое современная математическая физика? // Труды Математического института им. В. А. Стеклова РАН. 1999. Т. 226. С. 7-10.
2. Владимиров В.С. Что такое современная математическая физика? // Препринт, Математический институт им. В. А. Стеклова РАН. 2006. Т. МИ- АН №НС-06/001. С. 1-20.
3. Ландау Л.Д., Лифшиц Е. М. Квантовая механика. Нерелятивистская теория. М.: Физматлит, 2004. 800 с.
4. Кунин С. Вычислительная физика. М.: Мир, 1992. 518 с.
5. Давыдов А. С. Квантовая механика. СПб.: БХВ-Петербург, 2011. 704 с.
6. Бизли Д. Python. Подробный справочник. СПб.: Символ-Плюс, 2010. 864 с.
7. Марчук А. Х. Введение в Python для студентов-астрономов. Методическое пособие. СПб.: СПбГУ, 2016. 49 с.
8. Доля П. Г. Введение в научный Python. Харьков: ХНУ, 2016. 265 с.
9. Абрамовиц М., Стиган И. Справочник по специальным функциям с графиками, формулами и таблицами. М.: Наука, 1979. 832 с.
10. Сена Л. А. Единицы физических величин и их размерности. М.: Наука, 1977. 334 с.